

课程笔记

CS224N Lecture 8: Transformers

整理：AI Course Notes Contributors；授课：Chris Manning

2024 年春季

课程版本： 2024 年中文讲义整理版

笔记来源： AI Course Notes Contributors

本版处理： 移除课件截图和对应图注，不包含 Stanford 原始课件、图片或视频。

授权： 笔记依 CC BY-NC-SA 4.0 分享；完整许可与改动记录见下一页。

来源与许可

本中文笔记由 AI Course Notes Contributors 整理，课程版本为 2024；源稿：<https://github.com/hqhq1025/ai-course-notes/tree/717e2df65c3d2eee593c171156a30d4f8f6812b9/cs224n/lecture08>。

笔记内容依 CC BY-NC-SA 4.0 分享：<https://creativecommons.org/licenses/by-nc-sa/4.0/>。

本副本的改动：移除 Stanford 原始课件截图与依赖这些截图的图注，移除课程视频封面/链接，并清理 TeX 源稿中的行尾空白后重新编译为可独立阅读的 PDF；同时加入本来源与许可说明。完整许可文本随本文件夹提供。Stanford 课件、课件图片和视频不包含在该授权内；请从对应年份 Stanford 课程页访问原始课件。

目录

1 引言：为什么 Transformers 重要	3
1.1 Transformers 改变了什么	3
1.2 Transformer 不只属于 NLP	3
1.3 缩放律的启示	3
1.4 本章小结	3
2 从递归到 Attention	3
2.1 RNN 时代的主流做法	3
2.2 Attention 作为过渡	4
2.3 本章小结	4
3 理解 Transformer	4
3.1 完整架构	4
3.2 Self-Attention 的直觉	4
3.3 从公式到向量化	4
3.4 为什么 attention 还不够	5
3.5 Residual、LayerNorm 和 Scaled Dot-Product	5
3.6 位置编码：解决顺序信息缺失	5
3.7 多头注意力	5
3.8 Decoder 的两类 attention	6
3.9 Decoder 的收尾	6
3.10 本章小结	6
4 缺点与变体	6
4.1 Transformer 的主要缺点	6
4.2 位置表示的后续工作	7
4.3 高效 attention 变体	7
4.4 本章小结	7
5 总结与延伸	7
5.1 全讲回顾	7
5.2 课程结语	8
5.3 延伸阅读	8

1 引言：为什么 Transformers 重要

本讲的主题是 **Transformers**。这类模型建立在 self-attention 之上，已经成为现代 NLP 的默认架构，也逐渐扩展到图像、蛋白质、系统优化等领域。

本讲核心内容

1. Transformers 为什么在 NLP 中迅速取代 RNN
2. self-attention 的直觉、公式与向量化实现
3. Transformer encoder-decoder 的完整结构
4. positional encoding、multi-head attention、masked attention 的作用
5. Transformer 的局限，以及后续的高效变体

1.1 Transformers 改变了什么

过去几年里，Transformer 不只是提升了机器翻译的质量，更改变了整个 NLP 研究范式。其核心优势不是某一个小技巧，而是把序列建模的主干从 recurrence 改成了 attention。

1.2 Transformer 不只属于 NLP

本讲也强调了一个趋势：Transformer 不是只在文本任务里有效，而是已经扩散到更广泛的领域。幻灯片中给出了蛋白质折叠、图像分类和系统优化的例子。

1.3 缩放律的启示

幻灯片用 scaling laws 提出一个更大的问题：如果 Transformer 的性能会随着模型、数据和计算规模平滑上升，那么我们是否只需要继续扩大同样的架构？

本讲的判断

标题里问的是 “Is Attention All We Need?”，但本讲给出的答案更接近 “Attention 很重要，但还不够”。Transformer 的成功来自 attention + 位置编码 + 多头机制 + 残差 + layer norm + feedforward 的组合。

1.4 本章小结

Transformer 之所以重要，是因为它在训练效率、长距离依赖建模和大规模扩展性上都比经典 RNN 更适合现代深度学习工作负载。接下来要理解的是：它到底是怎么工作的。

2 从递归到 Attention

2.1 RNN 时代的主流做法

在 Transformer 之前，NLP 的主流方案是用双向 LSTM 编码输入，再用解码器逐步生成输出。这个范式虽然有效，但它有两个明显问题：难以并行，且长距离依赖的梯度传播路径太长。

RNN 的两个结构性瓶颈

1. **并行性差**：下一个状态必须依赖上一个状态，训练时很难充分利用硬件
2. **交互距离长**：两个相距很远的词要通过许多时间步才能彼此影响，信息和梯度都更难传播

2.2 Attention 作为过渡

上一讲已经看到，attention 可以让 decoder 直接访问 encoder 中任意位置的信息。Transformer 进一步把这个思想推广到编码器和解码器内部，变成 **self-attention**。

2.3 本章小结

从 RNN 到 attention 的关键转变是：不再让信息只能沿着时间轴一步一步流动，而是允许每个位置直接检索整个序列。Transformer 把这个思想推到极致，于是计算图也变得更适合并行硬件。

3 理解 Transformer

3.1 完整架构

Transformer 由 encoder 和 decoder 两部分组成。encoder 反复堆叠 self-attention 与 feedforward，decoder 则在 self-attention 之外，再增加一层 cross-attention 去读取 encoder 输出。

3.2 Self-Attention 的直觉

Attention 可以被理解成一个“fuzzy hashtable”：不是精确查找一个 key，而是根据 query 与所有 key 的匹配程度，返回一个加权和。

self-attention 的计算步骤

1. 对每个词计算 query、key、value
2. 用 query 和所有 key 计算 attention score
3. 对 score 做 softmax 归一化
4. 对 value 做加权求和，得到该词的新表示

3.3 从公式到向量化

对第 i 个位置，令

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i.$$

则 attention 权重为

$$\alpha_{ij} = \text{softmax}_j \left(\frac{q_i^\top k_j}{\sqrt{d_k}} \right),$$

输出为

$$o_i = \sum_j \alpha_{ij} v_j.$$

把整个序列堆起来后，向量化实现更简单：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

3.4 为什么 attention 还不够

如果只有 self-attention，模型本质上只是把 value 向量重新加权平均。这样虽然强大，但表达能力仍然不够，所以 Transformer 在每个 attention block 后面还加了 feedforward 层。

3.5 Residual、LayerNorm 和 Scaled Dot-Product

为了让深层 Transformer 可训练，原论文引入了三类训练技巧。

Residual Connection

残差连接把输入直接加到子层输出上：

$$x' = x + \text{Sublayer}(x).$$

它的作用是保留原始信息、帮助梯度传播，并让深层网络更容易学习到接近恒等映射的行为。

Layer Normalization

LayerNorm 在每个 token 的特征维度上做归一化，减少 layer 输入分布的漂移：

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

它比 batch norm 更适合 NLP，因为句子长度变化大，batch 的统计量也不稳定。

Scaled Dot-Product Attention

当 d_k 很大时， $q^\top k$ 的方差会变大，softmax 容易饱和。于是除以 $\sqrt{d_k}$ ，把数值尺度控制在更稳定的范围内。

3.6 位置编码：解决顺序信息缺失

self-attention 本身不关心词序。若直接把词向量送进去，“Man eats small dinosaur” 和 “Dinosaur eats small man” 的表示几乎没有顺序差别，这显然不对。因此需要把位置编码注入模型。

一种经典做法是把位置向量 p_i 加到 token 表示上：

$$x'_i = x_i + p_i.$$

原始 Transformer 使用正弦/余弦位置编码：

$$p_{i,2k} = \sin(i/10000^{2k/d}), \quad p_{i,2k+1} = \cos(i/10000^{2k/d}).$$

这种设计的优点是可外推到更长的序列，而且不同频率的正弦信号能编码相对位置信息。

3.7 多头注意力

单个 attention head 往往只能学到一种局部模式。多头注意力让模型并行地学习多个投影空间里的匹配关系。

若有 h 个头，第 ℓ 个头使用自己的参数：

$$Q^{(\ell)}, K^{(\ell)}, V^{(\ell)} \in \mathbb{R}^{d \times d/h}.$$

每个头独立计算 attention，再把输出拼接：

$$\text{MHAtt}(X) = Y[O^{(1)}; \dots; O^{(h)}].$$

直观上，不同的 head 可以分别关注词法邻近、句法依赖、长程共指等不同模式。

3.8 Decoder 的两类 attention

Transformer decoder 需要先做 masked self-attention，再做 encoder-decoder attention。前者防止“偷看未来”，后者从 encoder 的 memory 中读取源句信息。

$$e_{ij} = \begin{cases} q_i^\top k_j, & j < i, \\ -\infty, & j \geq i. \end{cases}$$

这保证了训练时可以并行计算所有位置，但每个位置仍然只能依赖左侧上下文。

Self-attention 与 cross-attention 的区别

- **Self-attention**: Q, K, V 都来自同一序列
- **Cross-attention**: Q 来自 decoder, K, V 来自 encoder

前者负责建模句内依赖，后者负责在生成时读取输入句子的记忆。

3.9 Decoder 的收尾

decoder 最后还会经过 feedforward、线性层和 softmax，变成下一个 token 的概率分布。

3.10 本章小结

Transformer 的主体可以概括为：位置编码 + 多头 self-attention + 残差连接 + layer norm + feedforward。encoder 和 decoder 的差异主要在于 decoder 多了 mask 和 cross-attention。

4 缺点与变体

4.1 Transformer 的主要缺点

即使 Transformer 很强，它仍然有两个广为人知的短板：第一，self-attention 的计算和显存开销对序列长度是二次的；第二，绝对位置编码未必是最好的顺序表示。

两个主要方向

1. 降低 $O(T^2)$ self-attention 的成本
2. 改进位置表示，让模型更好地利用顺序与结构信息

4.2 位置表示的后续工作

讲义里提到的几个方向包括 relative linear position attention、dependency syntax-based position 和 rotary embeddings。它们都试图让位置不只是一个静态的绝对 index，而是更贴近词与词之间的关系。

4.3 高效 attention 变体

为了摆脱 $O(T^2)$ ，研究者提出了许多高效 Transformer 变体。例如 Linformer 通过把序列长度维度投影到低维空间减少成本，BigBird 则用局部窗口、全局 token 和随机连接替代全连接注意力。

变体不一定真的更好

讲义最后提醒：很多 Transformer 修改并不会带来显著收益。也就是说，改结构并不天然意味着更强，是否 transfer 到实际任务中仍然需要实证验证。

4.4 本章小结

Transformer 的成功并不意味着它已经没有缺陷。它仍然面对 quadratic cost 和位置表示设计的问题。后续研究主要围绕两条线展开：一条是让 attention 更省算力，另一条是让位置编码更符合语言结构。

5 总结与延伸

5.1 全讲回顾

这节课从“attention 是否足够”的问题出发，最终给出一整套 Transformer 的机制：

- 用 self-attention 代替 recurrence，让交互距离降到 $O(1)$
- 用 positional encoding 把顺序信息重新注入模型
- 用 multi-head attention 捕捉不同类型的依赖
- 用 masked self-attention 保证 decoder 不偷看未来
- 用 cross-attention 让 decoder 访问 encoder 记忆
- 用 residual、LayerNorm 和 feedforward 让深层模型可训练

本讲的 takeaways

1. Transformers 不是只有 attention，而是 attention + 一整套配套设计
2. self-attention 的最大优势是并行性和短路径依赖
3. 位置编码是 Transformer 里不可或缺的一层
4. 多头机制让模型可以同时学习多个关系视角
5. 未来的改进主要集中在效率和位置建模上

5.2 课程结语

讲义最后的提醒也很直接：继续准备 assignment 4, 并关注后续的 project proposal。理解 Transformer 之后，下一步就是理解为什么 pre-training 能把模型性能再推高一个量级。

- “Good luck on assignment 4!”
- “Remember to work on your project proposal!”

5.3 延伸阅读

- Vaswani et al., Attention Is All You Need (2017): Transformer 的原始论文
- Shaw et al., Self-Attention with Relative Position Representations (2018): 相对位置编码
- Wang et al., Linformer / BigBird 等高效 attention 工作：降低 quadratic cost
- Vaswani 相关后续 lecture 和 CS224N 作业 4: Transformer 的实践训练